

Evaluation of source code in ACM ICPC style programming and training competitions

Wilson Julca-Mejia¹, Hugo D. Calderon-Vilca¹ and Flor C. Cárdenas-Mariño²

¹ Universidad Nacional Mayor de San Marcos, Lima 01, Perú
11200185@unmsm.edu.pe
hcalderonv@unmsm.edu.pe

² Universidad Nacional Tecnológica de Lima Sur, Lima 01, Perú
clavelyfcm@gmail.com

Abstract. The execution of potentially dangerous source code is one of the main problems in the platforms of programming contest. In this research we propose NinjaCoding, which is a web platform to evaluate the source code in programming contests and training ACM ICPC style. NinjaCoding is designed to be a scalable, safe and economical system, in addition our proposal ensures the execution of source code through the use of containers and subprocesses. Students can use NinjaCoding to participate in past or active contests and in training problems with live feedback. This system has been tested in several programming contestss that have taken place in various congresses and universities in different cities of Peru, qualifying more than 1000 presentations in the languages C / C ++, Python and Java. This document describes and discusses the architecture and the implementation that is used in NinjaCoding.

Keywords: Online judge, programming practice, auto grading, programming competition, programming contest, docker, sandbox.

1 Introduction

Annually, the ICPC sponsored by IBM is celebrated, which is an international programming contest where universities around the world participate. During the competition, the teams have about 5 hours to solve between 9 and 12 problems. Solutions must be programmed with C, C ++, Java or Python. The programs sent by the teams are compiled and executed with certain input data, if the program fails when calculating the solution, the team is notified of the error and can send the program again or try other problems [15]. The objective of the competitive programming is to produce skilled computer programmers who are much more prepared to produce better software or to deal with the most difficult problems in computer science research in the future [22].

In this type of programming contests the use of an evaluation system is an absolute necessity, because the required time to manually evaluate the source code can last for days [3].

Currently in Peru there are only some platforms for contests and training, each with scalability problems and aimed at a very small community. One of the platforms in Peru

is HuahCoding [16] that does not evaluate source code, but only evaluates the final response, therefore, it does not evaluate if there is compilation error, limit time exceeded or limit memory exceeded. Also, when a programming contest is organized in some events, external platforms such as Uva Judge [17], Timus Online Judge [18], among others, are used. The problem with these platforms is that they provide problems in the English language and therefore are not very suitable for students who are just starting college.

Programming contests are very important since they allow the introduction, popularization and promotion of computer science in all areas of society, especially in education [3].

One of the systems that was designed to support the programming contests was Mooshack. This system can be recognized as a pioneer in the automatic evaluation of web-based source code [6].

The manual evaluation in programming competitions that took place in the past lasted days, in addition the evaluation systems such as Mooshak, CMS, DOM judge and PC2 had deficiencies in completely managing the contests, because the collection and classification of shipments, handling of questions and clarifications of competitors, publication of results, and statistics were deficient [3].

Currently there are applications such as "CodeAcademy" "Rubymonk" and "Try Ruby" that teach specific programming languages and since they are specific they do not contribute much to develop algorithm design skills [5].

The different platforms of programming contests have a main concern: to guarantee the security in the execution of source code, to guarantee this, they use Virtual Machines [7] or Containers [6], and the containers do not incorporate their own kernel, but rather they run directly in the host's core. This shortens the execution route and remove the guest kernel and the virtual hardware layer. In addition, containers can share software resources, such as libraries, with the host, this way they can avoid the code duplication. The absence of kernel and some system libraries make the containers very light, the image sizes can be reduced to a few megabytes, which allows a fast boot process [10].

In this research we propose a platform called NinjaCoding that evaluates source code in different programming languages such as C / C ++, Python and Java style ACM ICPC and the way it is built provides scalability to add other compilers to evaluate presentations in more languages such as Ruby, Go , Php among others. In addition, this platform is not only aimed at participants of intermediate or advanced level but participants who are just beginning because we provide problems in Spanish and a repository of problems grouped by category and sorted by the level of difficulty.

For this research, first it was performed the analysis of modules of other programming platforms of programming competitions, then an architecture was designed, this architecture not only limit to programming and training contests, but also is used as a base to add other modules and functionalities in a way that is scalable and robust. After this design, the implementation stage was carried out. Here, a stack of technologies such as Python, Django, Redis, PostgreSQL, Docker, AngularJS, SocketIO was used. And finally it was validated in various online contests, such contests have been organized in various universities since 2016.

In section 1 the introduction is presented, in section II the state of the art is presented, in section III the architecture of the system is presented, in section IV the results and discussion are presented, and finally in section V the conclusion is presented.

2 Related word

Systems like Judge.org from [7] and Pythia from [5], provide security against malicious code for their SandBox use Virtual Machines with different approaches. The first one states it takes too long to restart the virtual machines after each correction, so they proceed to restart them after a dozen corrections to amortize the restart time. The second one states that using virtual machines that emulate hardware peripherals such as KVM or VirtualBox are too slow, this approach proposes a SandBox that runs a very short version of ArchLinux.

Another component are the tails (colas), which allows to manage the presentations in the Sandboxes and it is a common component in most of the judges of programming contests [1] [3] [4] [5] [6] [7], It is vital to ensure that a SandBox only takes care of one process at a time, whether it's compiling or executing a single presentation. These tails (colas), according to the approaches of each author, can prioritize certain processes over others; for example, the author [1] prioritizes the compilation processes because it's much faster and this way it provides a much faster feedback to the contestants.

In the architectures of judges of programming contests [1] [2] [3] [4] [5] [7], we observe modules with similar functionalities such as FrontEnd, Gradding, Queue Manager, Gradding, SandBox and Scoreboard . The FrontEnd is in charge of presenting the challenges or problems to the users and sending the presentations that are made by the contestants and passing them to the evaluation module (gradding). The Gradding controls the entire classification process by calling the compilers, the sandbox, and it compares the output of the solution with the expected response. The Queue Manager facilitates the evaluation of tasks in parallel, maintains a queue of presentations and distributes them among the SandBox. The Gradding processes the outputs of the executions and compilations made in the SandBox and generates various reports (for example, scorecards, detailed feedback reports for the contestants). The SandBox directs the presentation of the contestant in a controlled and secure environment, which limits its execution time, consumption of memory and calls to the system. The Scoreboard classifies the participants according to the number of solved problems and time of resolution.

3 Architecture of the evaluation system of the source code

This type of system is oriented to reinforce or study specific competitive programming topics and to simulate ACPC ICPC type contests.

The proposed architecture consists of the modules: FrontEnd, Manager, Grader, Queue Manager, SandBox, Problems Repository and Scoreboard.

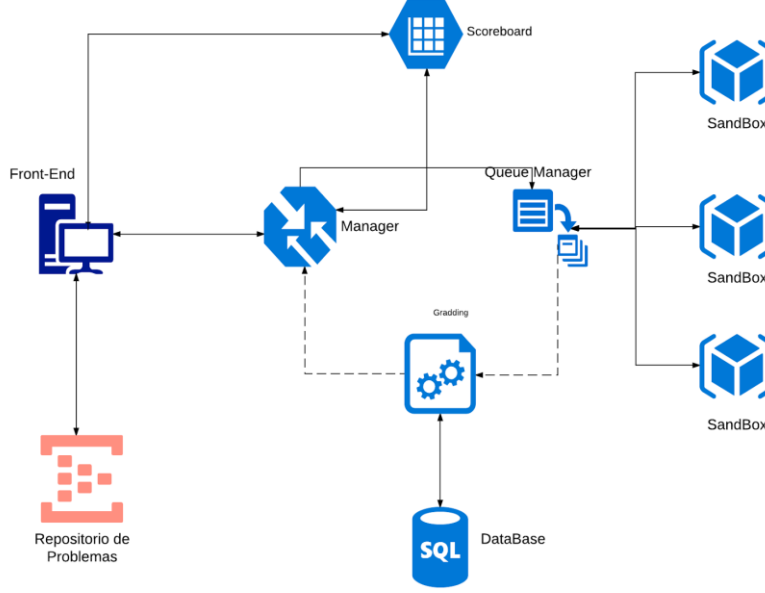


Fig. 1: Architecture of the evaluation system of the source code

3.1 SandBox

This component executes the presentation of a user in a controlled and secure environment, limiting its execution time, memory consumption and system calls [9]. A SandBox must ensure that the execution of a solution will not damage the system or the host computer, it must impose time restrictions, memory and network [3].

In our proposal to implement this component, we use Containers through Docker and subprocesses with the Python language.

We use subprocesses because it is a mechanism that allows to execute system commands in a secure way, to initiate processes, to communicate to their input pipes, output and error. Additionally we use Docker because it allows to isolate processes and control the resources that are going to be used in the execution and compilation of source code such as memory and CPU.

It is vital to have this module decoupled from the rest of the modules to ensure the robustness of the platform, because in this component is where the greatest risks will occur when executing or compiling source code that can be harmful to the host system.

In our research the SandBoxes are created from Docker images and these are created from Dockerfiles through subprocesses as shown in Table 1.

Dockerfile is a file that describes how a container image will be constructed, providing a set of instructions to simplify the construction process. All available commands are designed to create images for independent containers [6].

Table 1: Way to create an image from a SandBox

```
command = 'docker build -t {} {}'
subprocess.Popen(command.format(image_name, path_to_dockerfile).split(' '))
```

Each Dockerfile contains the commands to install the necessary compilers of each language supported by the system, this way it's possible to scaled to more gramming languages through this mecanism. For the creation of each SandBox we used, as base, an Alpine image that weighs only 5MB and has access to a package manager apk [19] as we see in Table 2.

Table 2: Contents of a Dockerfile file with the commands to install the compilers of C ++ and Python3

```
FROM alpine:3.4
RUN apk add --no-cache python3-dev
RUN apk add g++
```

To execute and compile, we use the docker run command, which is accompanied with the directory route where the source code is stored, the directory within the volume of the container where the command will be executed, the name of the image from which the container will be created, and the command to execute. Table 3.

Table 3: Execution and / or compilation of source code in a container

<code>docker run --rm -v {workdir}:/code -w /code {image} {command}</code>
--

The amount of memory that the container will use can be limited to the execution or compilation; an essential requirement for algorithmic problems in optimization, for this reason the -m parameter is added, followed by the maximum amount of memory to be used. This is used in problems where you need to evaluate the memory that uses some of the proposed presentations. Table 4.

Table 4: Execution or compilation of source code in a container with a memory limit of 15 Megabytes

<code>docker run -m 15M --rm -v {workdir}:/code -w /code {image} {command}</code>

The memory for the execution or compilation can be limited, as well as the execution time of the process in the container. To do this the timeout command comes first and then comes the command in Table 3, followed by the maximum amount of time to execute. This is useful for algorithmic problems where a time limit must be evaluated according to some algorithmic complexity that is posed by the problem setter. Table 5.

Table 5: Execution or compilation of source code in a container with a time limit of 1 second

<code>timeout 1 docker run --rm -v {workdir}:/code -w /code {image} {command}</code>
--

3.2 Queue Manager

This module is responsible for managing the sending of user presentations by sending either the SandBox or the Grader. It is the one that will know how many SandBox are available, and to which SandBox you have to send a certain presentation.

The Queue Manager not only sends the presentation of each user to the SandBox, but also the test cases of the problem and the constraints of the problem.

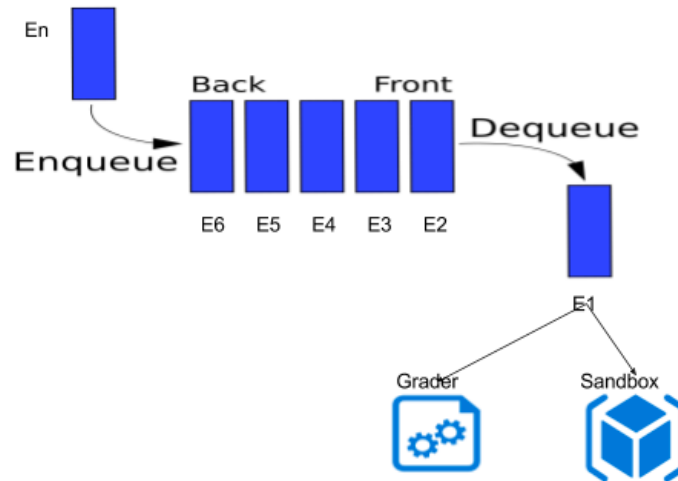


Fig. 2: Queue Manager and how it interacts with the Grader and SandBox modules

3.3 Grader

This module will be responsible for evaluating the output generated by the execution of a presentation of a participant with the expected output that has been posed by the problem setter and it emit a result, this result can be either of success, incorrect answer, correct answer, time limit exceeded, memory limit exceeded or execution error. This module will also evaluate if there were compilation errors at the time of compiling the code, this will be also a result that will be reported to the user of said presentation. Once evaluated, the presentation of the participant, the Grader notifies the Manager that there is a evaluated presentation ready to be shown to the participant. The evaluation is carried out by comparing the JSON which was generated after the execution or compilation of the source code with the expected JSON. This can be seen in Table 6.

Table 6: JSONs to evaluation

JSON Generado	JSON Esperado
<pre>{ "completed": Bool, "stderrG": String, "stdoutG": String, "timeG": Double, "memoryG": Double }</pre>	<pre>{ "stdoutE": String, "timeE": Double, "memoryE": Double }</pre>

The evaluation process is based on the algorithm of Figure 3. It start by evaluating the stderrG, if it is different from empty, it then there was a compilation error. In this case

the state and the generated error are saved to CE, then it is evaluated if the evaluation process is complete, if it is, then it is sent to the Queue Manager for its eventual execution; if it's not, then we proceed to evaluate if the time and memory limits have been exceeded, if they haven't, then we proceed to save the TLE or MLE states; if they have, then the output cases that have been generated are finally evaluated. If this generated output is equal to the expected output it means that the proposed presentation is correct and the AC state is saved in the database, if they are not equal then the WA state is saved. The meaning of each of these abbreviations can be seen in Table 7.

Table 7: Response states that the Grader module emit

Abreviatura	Significado
CE	Compiler Error. It means that there is a compilation error.
TLE	Time Limit Exceeded. It means that the limit time for the problem has not been exceeded. Possibly because the algorithm is not optimal or the complexity is very high.
MLE	It means that the limit memory has not been exceeded, possibly because too much memory is being used in arrays
WA	Wrong Answer. It means the answer is wrong.
AC	Accepted. It means that all the limits have been exceeded and the answer is correct and optimal.
RTE	Run Time Error. It means that there was a problem in the execution of its presentation.

In the evaluation process, if there was an error in its execution, the state of RTE is saved. This evaluation, depending on whether the presentation was made in a compiled or interpreted language, is made in the second or first time you enter the Grader.

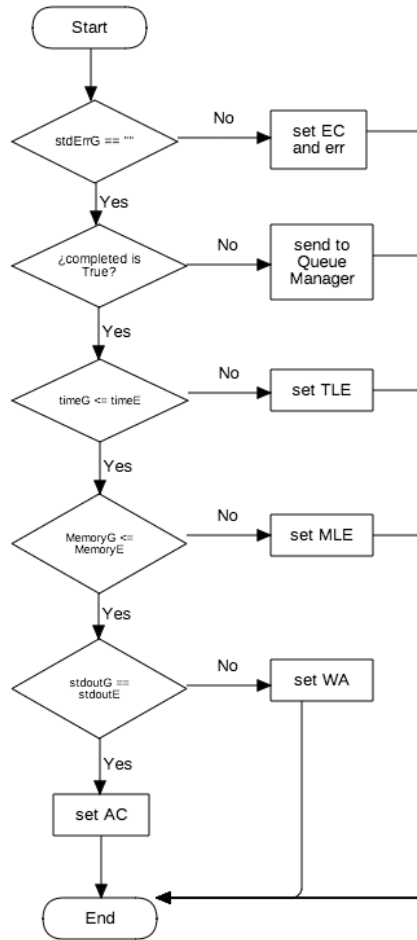


Fig. 3: Evaluation algorithm for a compiled language.

3.4 Manager

The Manager is the central module in the entire architecture, it is the one that has communication with most of the modules. It receives the presentation of a user through the FrontEnd, sends the presentation to the Queue Manager, receives the notification of the Grader when a solution has already been evaluated, notifies the Scoreboard for its update and sends the response to the user through FrontEnd. Figure 4.

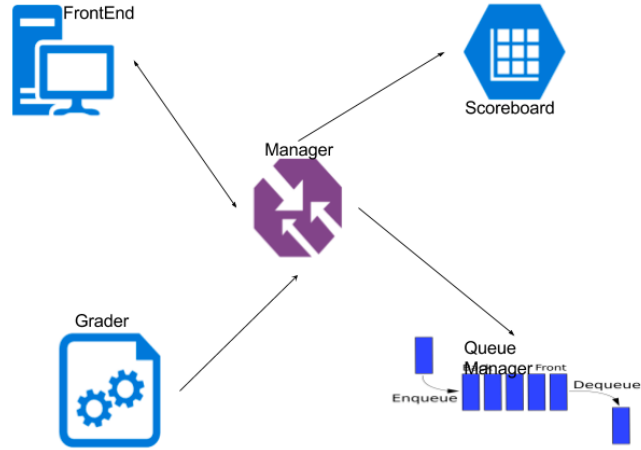


Fig. 4: Interaction of the Manager with other modules

3.5 Scoreboard

This module is responsible for showing the results of all the teams in merit order. The best qualified are at the top and goes downwards decreasing to the worst ones. Figure 5.

This classification is made according to 3 variables:

1. The number of solved problems
2. How long each problem has been solved
3. The number of attempts for each solved problem

For each failed attempt there is a penalty of 20 minutes, so if a problem is solved on the 2nd attempt at 30 minutes, it is as if it had been resolved at 50 minutes.

Rank	Equipo	A	B	C	D	E	F	G	H	Solved	Time
1	UPC - INSANECodeOverride	1	2	1	5	1	1		4	6	498
2	UNI - 1000KB	6	8	1	4	1	1	2	1	6	537
3	UNI - 2000KB		8	1	2	1	1	5	1	5	249
4	UNI - MDEVELOP	3	2	1		1	1		1	5	588
5	UNI - AhoritaNoJoven	1	13	1	11	1	1		1	4	131
6	UNMSM - JavaHaters	3	3	1	2	1	2		1	4	213
7	UNJFSC - 2MYR			2	8	1	1		1	4	249
8	UNMSM - SP11			2		1	1	1	3	4	331
9	UNU - TheBuggers	2		4		2	1	1	1	3	397
10	UL - DrinkTeam	2		2		1	5			0	0
11	UNP - sout	1	4					11	4	0	0

Fig. 5: Scoreboard of the 2016 CONEISC programming contest.

3.6 FrontEnd

This module is responsible for showing the results to the participant, of each presentation that is sent to the system, showing all the problems Figure 6, showing all the contests, showing an interface for the solution-sending, showing the user panel, showing functionalities to the problem setter and problem tester.

The screenshot shows the NINJACODING website interface. At the top, there's a navigation bar with 'HOME', 'TRAINING', 'CONTEST', and 'RUN STATUS' links, along with a 'Get Started' button. The main content area is divided into two columns. The left column contains a sidebar with 'OPTIONS' (SCOREBOARD, CLASIFICACIONES), 'SUBMIT' (G++ 5.4, Upload Solution, ENVIAR), and 'TOP SCORES' (juan, 22). The right column displays the problem details for 'Trivial'. It shows a difficulty level of 2.0 seconds and 51 submissions. The description explains the concept of 'triviality' of a number N as the ratio of the sum of its proper divisors to N. The input format specifies the number of test cases T, followed by pairs of integers A and B. The output format requires printing the number N such that $A \leq N \leq B$ and N has the minimum triviality. Sample input and output are provided at the bottom.

SAMPLE INPUT	SAMPLE OUTPUT
2	7
7 7	25
24 28	

Fig. 6: Visualization of a problem in a contest.

3.7 Problems Repository

This module shows all the problems used in past programming contests created by the community. The way the presentations are grouped is by categories, ordered by level of difficulty and showing statistics of how many solution attempts there are and how many ones have been satisfactory for each problem.

This module would be one of the access points for the training of the participants according to the subject and difficulty they wish to reinforce.

4 Results and discussion

The data in our platform is provided by the problem setters and these can create their own problems or obtain from other sources such as the repositories of past programming contests of the ACM ICPC. All these problems are tested through 4 tests: correct answer test, incorrect answer test, compile error test and exceeded limit time test. After passing these tests, the problems can be used either in contests or become public in the platform's problem repository.

The testing of our system of source code evaluation has been carried out in two stages: In the first stage, 4 problems of the practice contest of the East Central North America 2015 competition were used [20]. Table 9.

Table 9: test problems

<i>Problem</i>	<i>Title</i>
A	Bottled-Up Feelings
B	Mastering Master-mind
C	Shuffling Along
D	Towering Problem

The results of the test of these 4 problems were from the 16 carried out tests, the 16 showed the expected evaluation. Figure 7.

 HOME PUZZLES CONTEST COMMUNITY Get Started				
username	Problem title	Verdict	Language	Datetime
pytuxi	Bottled-Up Feelings	Accepted	C++	July 26, 2016, 8:11 p.m.
pytuxi	Bottled-Up Feelings	Time limit	C++	July 26, 2016, 7:56 p.m.
pytuxi	Bottled-Up Feelings	Compiler error	C++	July 26, 2016, 7:55 p.m.
pytuxi	Bottled-Up Feelings	Wrong answer	C++	July 25, 2016, 7:54 p.m.
pytuxi	Mastering Mastermind	Accepted	C++	July 25, 2016, 7:53 p.m.
pytuxi	Mastering Mastermind	Time limit	C++	July 25, 2016, 7:51 p.m.
pytuxi	Mastering Mastermind	Compiler error	C++	July 25, 2016, 7:50 p.m.
pytuxi	Mastering Mastermind	Wrong answer	C++	July 25, 2016, 7:49 p.m.
pytuxi	Shuffling Along	Accepted	C++	July 25, 2016, 7:45 p.m.
pytuxi	Shuffling Along	Time limit	C++	July 25, 2016, 7:43 p.m.
pytuxi	Shuffling Along	Compiler error	C++	July 25, 2016, 7:42 p.m.
pytuxi	Shuffling Along	Wrong answer	C++	July 25, 2016, 7:39 p.m.
pytuxi	Towering Problem	Accepted	C++	July 25, 2016, 7:37 p.m.

Fig. 7: Results status of the 4 tests for each problem

In the second phase, since 2016, NinjaCoding was tested in several contests in congresses and in different universities and it has graded more than 1000 presentations in different programming languages that are detailed in Table 10.

Tabla 10: Test results in different events

Event	University	Number of evaluated presentations	Number of participants
CONEISC 2016 (National Congress of Systems and Computer Engineering Students)	Ucayali national university	224	45 students from different universities
Selection contest UNI-FIIS Stage I 2016	Ingeniering national university	97	22 students of UNI
Selection contest UNI-FIIS Stage II 2016	Ingeniering national university	74	20 students of UNI
Contest in the UNMSM-FISI anniversary, 2016	Mayor de San Marcos national university	125	39 students between UNI and UNMSM

In our research as well as the author [6], the Sandbox module was built with Docker, but we are different on the fact that when running Docker we use threads of the Python language. Python threads are a safe way to start processes from the Python language and have a communication method that allows the sending of standard data input (stdin) and read or output data (stdout) and error (stderr) without risk of interlocks [21].

Building SandBox using Virtual Machines the way [7] does it in its Judge.org project, makes the boot of the sandbox much slower, instead in our research we use Docker for the SandBox, where they directly use the host's core which shortening the execution route of system calls, eliminate the guest kernel and the virtual hardware layer that virtual machines [10] use, therefore, it is much faster.

We agree that [4], the resolutions of algorithmic problems should not be forced to be solved in a single programming language, for this reason, in this investigation, the posed problems, both in contests and trainings, can be solved with different programming languages.

The research [3] is limited to support traditional programming languages such as C / C ++, Java and Pascal, whereas in this research we have taken care of supporting also popular and modern programming languages such as Python and Ruby.

5 Conclusions

In this article we presented the architecture and platform of the automatic evaluation system for source code that is used for both programming contests and training. In addition, it was possible to support the required languages for ACPC ICPC type programming contests and to provide scalability to extend the use of other programming languages.

Automatic systems for evaluation acquire more importance thanks to the growing demand for IT programmers. An evaluation system can greatly improve the quality of its solutions for participants, since participants find themselves forced to produce efficient and error-free solutions; in the educational environment, this type of systems also increases the interest and academic performance of students, increasing from 40% to 70% [11].

The most important requirement for the automatic evaluation must be the safe and isolated execution environment of the other modules of the system, since the source code that is being executed can be harmful and dangerous. NinjaCoding was built to meet these requirements for automatic evaluation. Through Docker and Python language threads it ensures the isolation of the execution environment and provides flexibility to add constraints of time and memory and support more programming languages.

Our proposed system has a promising development potential, because we are going to give support to more programming languages according to the tendencies of use of these, we will also develop the educational module so that this platform can be applied in the different programming courses in the various universities. Finally we will provide support to different types of contests such as IOI [23] and Google Code Jam [24] so that there is more diversity of contests.

References

1. S. Maggiolo y G. Mascellani. Introducing CMS: A Contest Management System. Olympiads In Informatics 6, 86 – 99 (2012).
2. J. Fernando y M. M. Liem. Componentes And Architectural Design of an Autograder System Family. Olympiads In Informatics 8, 69 – 79 (2014)
3. M. Jovanov, B. Kostadinov, E. Stankov, M. Mihova y M. Gusev. State Competitions in Informatics and the Supporting Online Learning and Contest Management System with Collaboration and Personalization Features MENDO. Olympiads In Informatics 7, 42 – 54 (2013)
4. L. H. Chávez, A. Gonzáles y J Ponce. omegaUp: Cloud-Based Contest Management System and Training Platform in the Mexican Olympiad in Informatics. Olympiads In Informatics 9, 169 – 178 (2014)
5. S. Combéfis y V. Clément. Teaching Programming and Algorithm Design with Pythia, a Web-Based Learning Platform. Olympiads In Informatics 6, 31 – 43 (2012)
6. F. Spacek, R. Sohlich y T. Dulík. Docker as Platform for Assignments Evaluation. Procedia Engineering 100, 1665-1671 (2015)
7. O. Giménez, J. Petit, S. Roura. Judge.org: An educational Programming Judge. Proceedings of the 43rd ACM technical symposium on Computer Science Education, 445 – 450 (2012)
8. D. J. Malan. CS50 Sandbox: Secure Execution Of Untrusted Code. Proceeding of the 44th ACM technical symposium on Computer science education, 2013.
9. M. Mares y B. Blackham. A New Contest Sandbox. Olympiads in Informatics 6, 100 – 109 (2012)
10. T. Combe, A. Martin y R. Di Pietro. To Docker or Not to Docker: A Security Perspective. IEEE Cloud Computing 3 (5), 54 – 62 (2016)
11. H. Wu, Y. Liu, L. Qiu y Y. Liu. Online Judge System and Its Applications in C Language Teaching. International Symposium on Educational Technology (2016)
12. D. Bernstein. Containers and Cloud: From LXC to Docker to Kubernetes . IEEE Cloud Computing 1 (5), 81 – 84 (2014)
13. W. Felter, A. Ferreira, R. Rajamony, J. Rubio. An Updated Performance Comparison of Virtual Machines and Linux Containers. IBM Research Report (2014)
14. M. Shams Ul haq, L. Liao, M. Lerong. Design and Implementation of Sandbox Technique for Isolated Applications (2016)
15. ACM ICPC. Avalaible: <https://icpc.baylor.edu/compete/problems> [Accessed: Nov. 3, 2017]
16. HuahCoding. Avalaible: <http://huahcoding.com> [Accessed: Oct. 3, 2017]
17. Uva Judge. Avalaible: <https://uva.onlinejudge.org/> [Accessed: Oct. 3, 2017]
18. Timus Judge Online. <http://acm.timus.ru/> [Accessed: Oct. 3, 2017]
19. Alpine Linux. Avalaible <https://alpinelinux.org/> [Accessed: Feb. 10, 2016]
20. East Central North America Region 2015. Avalaible: <http://acm-ecna.yzu.edu/PastResults/2015/problemset.html> [Accesed: Dec, 15, 2015]
21. Subprocesos. Avalaible: <https://www.python.org/dev/peps/pep-0324/> [Accesed: Nov, 20, 2016]
22. S. Halim y F. Halim, Competitive Programing 2, pp 1 (2011)
23. IOI. Avalaible: <http://www.ioinformatics.org/index.shtml> [Accessed: Nov. 3, 2017]
24. Google Code Jam: <https://code.google.com/codejam/> [Accessed: Nov. 3, 2017]